Parcel Box Planner - about 30 minutes

STATUS: proposed task. Submit Studio_Proposal.txt and get this idea approved before recording. This guide is checked against the bundled inputs; approval and an actual recording are not established by the guide.

Plan: about 29:45 including the closing reflection; allow a natural range of roughly 28-32 minutes. Rehearse once on a disposable copy to establish your own duration. Do not pad the take, rush your speech or skip tests to match the clock.

Read aloud only the complete words inside the quotation marks after SAY. Do not read block labels, timings, code, ACTION, EDIT, CHECK or IF WRONG. A result SAY is used only after its preceding CHECK matches the screen. The optional error branches apply only to their stated, observed problem.

Before recording - off screen

ACTION	Complete Preflight_Checklist.txt. Prepare ordinary windows with VS Code on the left and Firefox on the right, without covering each other. Keep that arrangement for the entire take. Do not use macOS full-screen Split View or maximize Firefox over the editor. Open Goal.pdf in the right-hand Firefox window. Prepare the neutral Box_Record folder in the left-hand VS Code window, but do not inspect or repair the task yet. Show absolute line numbers and word wrap; disable format-on-save/type and automatic quote/bracket closing. Keep the guide on paper or another device. Confirm the local HTML opens in native Firefox on the Mac.

1. Intro - 0:00-0:15

ACTION	Start TaskRecorder and immediately minimize its window. Leave Goal.pdf visible in Firefox on the right, with the ordinary VS Code window on the left. Keep the introduction under 20 seconds.
SAY	"I will be beginning my task soon. Here is the workflow I have in mind."

2. Read the packing policy - 0:15-3:30

SAY	"I am minimizing the Goal PDF window with Firefox's yellow button at the upper left of that window. I am then left-clicking Visual Studio Code in the side Dock to begin reviewing the task."
ACTION	Minimize the Firefox window; activate the already prepared VS Code window without changing its size.
SAY	"In Visual Studio Code's Explorer, on the left side of the editor, I am left-clicking input 2.md. I want to understand the packing policy before changing any calculations."
ACTION	Open the Markdown source, not its rendered preview. Point to the Packing policy heading near the top.
SAY	"The plan must pack every item into identical boxes. Full boxes counts only completely filled boxes. Items in partial box is the remainder after those full boxes, so an exact fit must have a remainder of zero."
ACTION	Point briefly to the relevant policy bullets. Read the policy as a connected explanation; do not point to every source line.
SAY	"Boxes needed includes one extra box when a remainder exists. Total slots is the number of boxes multiplied by the capacity of one box. Unused slots is that total capacity minus the number of items, and it must never be negative for valid inputs."
SAY	"Below the packing policy, the browser-validation section requires whole numbers. Items to pack must be from one through ten thousand. Items per box must be from one through five hundred. I will preserve those existing field rules."
ACTION	Point to Existing browser validation, below the policy.

SAY	"I am scrolling down inside the input 2.md editor to the Seven acceptance groups heading. P01 through P06 check the five result rows. P07 checks three invalid inputs and then restores a valid plan. I will complete all seven groups."
ACTION	Scroll to the acceptance groups. Briefly locate P01, P02 and P07 without editing the brief.
SAY	"The first two groups are especially useful together. Twenty-three items in boxes of six needs a partially filled fourth box. Twenty-four items in boxes of six fits exactly into four boxes. The planner must distinguish those cases."

3. Inspect and preserve the starter - 3:30-5:00

SAY	"In Visual Studio Code's Explorer on the left, I am left-clicking input 1.html. I am pressing Control-G, entering 47 in the Go to Line box near the top of the editor, and pressing Return to inspect calculatePacking."
ACTION	Use Control-G, type 47 and Return. The function begins on physical line 47. Keep lines 49-55 visible, using a small downward scroll within the editor if necessary.
SAY	"In input 1.html, line 51 already rounds the number of full boxes down correctly. The four statements on lines 52 through 55 calculate the partial-box items, boxes needed, total slots and unused slots. Those are the statements I will compare with the policy."
ACTION	Point to line 51, then the four statements at 52-55. Do not edit the starter.
SAY	"In Visual Studio Code's File menu at the top of the screen, I am left-clicking Save As to create a separate working copy."
ACTION	Open File > Save As. Leave the destination at the Box_Record folder containing the starter.
SAY	"In the Save As dialog over Visual Studio Code, I am left-clicking the filename field near the top and pressing Command-A to enter final output 1.html. I am keeping the recording folder as the destination and left-clicking Save in the lower-right corner."
ACTION	Save the output in Box_Record. Confirm the output tab name and both unchanged inputs in Explorer.

4. Show two incorrect baseline results - 5:00-7:30

SAY	"I am left-clicking Firefox in the side Dock to restore its window on the right. In Firefox's File menu at the top of the screen, I am left-clicking Open File."
ACTION	Restore the prepared right-hand Firefox window and open File > Open File.
SAY	"In Firefox's Open File dialog, I am selecting final output 1.html from the recording folder, then left-clicking Open in the lower-right corner. The working copy is still unmodified."
ACTION	Open the output copy, not input 1.html. Keep Firefox in its prepared right-hand position. The two fields and complete result table should fit at the rehearsed zoom.
SAY	"For the partial-box baseline, I am left-clicking Items to pack, the top field below the note, and pressing Command-A to enter 23. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes immediately below the second field."
ACTION	Enter 23 / 6, submit and inspect all five table rows below the button.

CHECK	Starter result, top to bottom: Full boxes 3; Items in partial box 20; Boxes needed 3; Total slots 9; Unused slots 14. Read the next SAY only if those actual values appear.
SAY	"The table below Calculate boxes shows three full boxes, but it also reports twenty items in a partial box, only three boxes needed, nine total slots and fourteen unused slots. Those values do not describe packing twenty-three items into boxes of six."
SAY	"The correct plan is three full boxes and five items in a fourth box. Four boxes provide twenty-four slots, leaving one unused slot. I will use that expected result when I repeat P01 after the repair."
SAY	"For the exact-fit baseline, I am left-clicking Items to pack at the top of the form and pressing Command-A to enter 24. Items per box, directly below it, remains 6. I am left-clicking Calculate boxes below the fields."
ACTION	Change only Items to pack to 24 and submit.
CHECK	Starter result: 4 / 20 / 4 / 10 / 14.
SAY	"For the exact fit, the table reports four full boxes and four boxes needed, but the partial-box row still shows twenty. Total slots is ten and unused slots is fourteen. The correct exact-fit result has no partial-box items, twenty-four total slots and zero unused slots."

5. Repair four short statements - 7:30-12:30

SAY	"I am left-clicking the final output 1.html tab at the top of Visual Studio Code's left-hand editor window. I am pressing Control-G, entering 52 and pressing Return to reach the first statement below the full-box calculation."
ACTION	Activate the output tab in VS Code. Go to physical line 52. Verify the filename before editing. For each edit, select only the statement from const through its semicolon, preserving the existing newline and indentation.
SAY	"In final output 1.html on line 52, I am selecting the partialItems statement from const through its semicolon. I am replacing the subtraction with item count remainder box size, using the percent operator. The remainder counts the items left after completely filled boxes."
EDIT	Replace the statement on line 52 with: `const partialItems = itemCount % boxSize;`
CHECK	Only one physical line was replaced. The statement on line 51, const fullBoxes = Math.floor(itemCount / boxSize);, remains unchanged.
SAY	"Line 51 still uses Math.floor for completely filled boxes. Line 52 now uses the remainder operator for the partial box. Those results describe different parts of the same order, so I am keeping both."
SAY	"In final output 1.html on line 53, directly below partialItems, I am selecting the boxesNeeded statement and changing Math.floor to Math.ceil. Rounding up includes the last box when some items remain, while an exact whole-number quotient stays unchanged."
EDIT	Replace line 53 with: `const boxesNeeded = Math.ceil(itemCount / boxSize);`
CHECK	Physical lines 54 and 55 have not moved.
SAY	"With twenty-three items and a capacity of six, the quotient is greater than three, so the planner needs four boxes. With twenty-four items, the quotient is exactly four. Math.ceil keeps that exact-fit case at four rather than adding an unnecessary fifth box."

SAY	"In final output 1.html on line 54, directly below boxesNeeded, I am selecting the totalSlots statement and changing addition to multiplication. The total capacity is the number of boxes multiplied by the number of items one box holds."
EDIT	Replace line 54 with: `const totalSlots = boxesNeeded * boxSize;`
SAY	"For four boxes that each hold six items, multiplication gives twenty-four slots. Adding four and six would give ten, which is not the combined capacity. I am keeping the units consistent with the Total slots label."
SAY	"In final output 1.html on line 55, directly below totalSlots, I am selecting the unusedSlots statement and reversing the subtraction. It must be total slots minus item count, so the result counts empty spaces after all items have been packed."
EDIT	Replace line 55 with: `const unusedSlots = totalSlots - itemCount;`
SAY	"For twenty-three items and twenty-four slots, the unused space is one slot. For an exact fit, the subtraction is zero. A valid plan should not report negative unused space because the box count has already been rounded up."
SAY	"I am checking lines 52 through 55 in final output 1.html, immediately below the unchanged full-box formula. In Visual Studio Code's File menu at the top, I am left-clicking Save to save the four repaired statements."
ACTION	Confirm the four exact EDIT statements and save. Preserve all other JavaScript, native field attributes, labels and markup. Do not format the file or insert/delete physical lines.
CHECK	Output lines 52-55 match the EDIT blocks, and the output tab has no unsaved-change dot.

6. Prepare the acceptance checks - 12:30-13:00

SAY	"In Visual Studio Code on the left, I am left-clicking input 2.md in the top tab bar and scrolling to P01 under Seven acceptance groups. I am keeping Firefox visible on the right so the brief and the calculated results stay readable together."
ACTION	Show the brief beside Firefox. Both windows remain in their prepared positions; neither covers the other.
SAY	"In Firefox on the right, I am left-clicking Reload beside the address bar to load the saved final output 1.html. I will replace the field values for each group and compare all five table rows with the brief."
ACTION	Reload the exact output copy. Each following result SAY is conditional on the preceding CHECK matching.

7. P01 - partial final box - 13:00-14:30

SAY	"For P01 in Firefox, I am left-clicking Items to pack, the top field below the note, and pressing Command-A to enter 23. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes below the second field."
ACTION	Enter 23 / 6 and submit. Compare every result row with P01 in the visible brief.
CHECK	Full boxes 3; Items in partial box 5; Boxes needed 4; Total slots 24; Unused slots 1.
SAY	"Reading the table below the button, Full boxes is three, Items in partial box is five, Boxes needed is four, Total slots is twenty-four and Unused slots is one. All five results match P01."

SAY	"Three full boxes hold eighteen items. The remaining five items need a fourth box, and one of its six slots is unused. The repaired results describe every item without losing the partial box."

8. P02 - exact fit - 14:30-16:00

SAY	"For P02 in Firefox, I am left-clicking Items to pack at the top and pressing Command-A to enter 24. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 24 / 6 and submit; compare all five rows with P02.
CHECK	4 / 0 / 4 / 24 / 0.
SAY	"The table now shows four full boxes, zero items in a partial box, four boxes needed, twenty-four total slots and zero unused slots. All five results match the exact-fit group."
SAY	"Every box is full in this case. The zero remainder prevents a false partial-box result, and rounding the exact quotient up does not create an extra box. The total capacity equals the item count."

9. P03 - one item - 16:00-17:30

SAY	"I am moving to Visual Studio Code's input 2.md editor on the left and scrolling down to P03 and P04. I am then returning to Firefox on the right to check the smaller quantities."
ACTION	Activate and scroll the brief only; keep both windows visible. Return focus to Firefox by the next field click.
SAY	"For P03 in Firefox, I am left-clicking Items to pack, the top field, and pressing Command-A to enter 1. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 1 / 6 and submit; compare all five rows with P03.
CHECK	0 / 1 / 1 / 6 / 5.
SAY	"The table shows zero full boxes, one item in a partial box, one box needed, six total slots and five unused slots. All five results match P03."
SAY	"A positive order still needs a box even when it cannot fill one. This check confirms that rounding up does not lose a small order and that the remaining capacity is counted as empty space."

10. P04 - one item per box - 17:30-19:00

SAY	"For P04 in Firefox, I am left-clicking Items to pack at the top and pressing Command-A to enter 9. I am left-clicking Items per box directly below it and pressing Command-A to enter 1. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 9 / 1 and submit; compare all five rows with P04.
CHECK	9 / 0 / 9 / 9 / 0.
SAY	"The table shows nine full boxes, zero items in a partial box, nine boxes needed, nine total slots and zero unused slots. All five results match P04."
SAY	"A capacity of one is valid. Each item fills a separate box, so the full-box count and total box count both equal nine. There is no remainder and no empty space."

11. P05 - box larger than the order - 19:00-20:30

SAY	"In Visual Studio Code's input 2.md editor on the left, I am scrolling down to P05 and P06. I am returning to Firefox on the right to compare those two sizes."
ACTION	Activate and scroll the brief; retain the window positions. Return focus to Firefox by the next field click.
SAY	"For P05 in Firefox, I am left-clicking Items to pack, the top field, and pressing Command-A to enter 7. I am left-clicking Items per box directly below it and pressing Command-A to enter 20. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 7 / 20 and submit; compare all five rows with P05.
CHECK	0 / 7 / 1 / 20 / 13.
SAY	"The table shows zero full boxes, seven items in a partial box, one box needed, twenty total slots and thirteen unused slots. All five results match P05."
SAY	"The box can be larger than the entire order. The planner keeps all seven items in one partial box, rather than treating the unused space as additional items or reporting zero boxes."

12. P06 - large valid order - 20:30-22:00

SAY	"For P06 in Firefox, I am left-clicking Items to pack at the top and pressing Command-A to enter 10000. I am left-clicking Items per box directly below it and pressing Command-A to enter 64. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 10000 / 64 and submit; compare all five rows with P06.
CHECK	156 / 16 / 157 / 10048 / 48.
SAY	"The table shows one hundred fifty-six full boxes, sixteen items in a partial box, one hundred fifty-seven boxes needed, ten thousand forty-eight total slots and forty-eight unused slots. All five results match P06."
SAY	"The item count is at its allowed maximum. One hundred fifty-six full boxes hold nine thousand nine hundred eighty-four items. The remaining sixteen items fit into one more box, which leaves forty-eight spaces unused. The larger input follows the same rules as the small orders."

13. P07 - invalid inputs and recovery - 22:00-25:30

SAY	"In Visual Studio Code's input 2.md editor on the left, I am scrolling down to P07, Native validation and recovery. I am returning to Firefox on the right to check each blocked submission and then a valid recovery."
ACTION	Activate and scroll the brief; retain both window positions. Return focus to Firefox by the next field click.
SAY	"For the first P07 check in Firefox, I am left-clicking Items to pack, the top field, and pressing Command-A followed by Backspace to leave it empty. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes."
ACTION	Clear Items to pack, set capacity 6 and submit. Point to the native browser prompt at the top field.
CHECK	Native required-field validation blocks submission at Items to pack. The previous P06 table values remain unchanged.

SAY	"The browser asks for the missing Items to pack value at the top field. The table still contains the previous large-order result. The page calls that the last successful calculation; it is not a new plan for the empty field."
SAY	"For the second P07 check in Firefox, I am left-clicking Items to pack at the top and pressing Command-A to restore 23. I am left-clicking Items per box below it and pressing Command-A to enter 0. I am left-clicking Calculate boxes."
ACTION	Enter 23 / 0 and submit. Point to the native prompt at Items per box.
CHECK	Native range validation blocks Items per box because 0 is below its minimum of 1. No new calculation runs; P06 remains visible.
SAY	"The browser blocks Items per box because zero is below its minimum of one. A box must have a positive capacity. The old table has not been recalculated, so the calculation never divides by a zero capacity."
SAY	"For the third P07 check in Firefox, I am left-clicking Items per box, the lower field, and pressing Command-A to restore 6. I am left-clicking Items to pack above it and pressing Command-A to enter 2.5. I am left-clicking Calculate boxes."
ACTION	Enter 2.5 / 6 and submit. Point to the native validation prompt at Items to pack. Browser wording can vary; the required observation is a blocked step mismatch.
CHECK	Items to pack has a native step mismatch because 2.5 is not a whole number. No new calculation runs; P06 remains visible.
SAY	"The browser blocks the Items to pack field because two point five is not a whole-number item count. The previous plan remains visible, and no new plan has been calculated for the fractional value."
SAY	"For the recovery check in Firefox, I am left-clicking Items to pack at the top and pressing Command-A to restore 23. I am left-clicking Items per box directly below it and pressing Command-A to enter 6. I am left-clicking Calculate boxes below the fields."
ACTION	Enter 23 / 6 and submit; inspect every row again.
CHECK	Recovery: 3 / 5 / 4 / 24 / 1. The old large-order values are replaced by the valid new plan.
SAY	"The table returns to three full boxes, five items in a partial box, four boxes needed, twenty-four total slots and one unused slot. The valid plan works again after all three blocked submissions. P07 is complete."

14. Final saved-file review - 25:30-26:15

SAY	"I am returning to Visual Studio Code on the left and left-clicking final output 1.html in the top tab bar. I am pressing Control-G, entering 51 and pressing Return to inspect the saved calculation."
ACTION	Open the output tab and show physical lines 51-55. Confirm that it has no unsaved dot and the two inputs remain separate in Explorer.
SAY	"In final output 1.html, line 51 keeps the original full-box calculation. Lines 52 through 55 now use the remainder, round the total box count up, multiply for total slots and subtract items from total slots. The output is saved, and both original inputs remain separate."

15. Closing reflection - 26:15-29:30

CHECK	Read the following reflection only if the preceding checks actually passed. Rehearse at a clear natural pace for 3-4 minutes, never over four. Do not invent an error or read a claim about a test you did not perform.
SAY	"I completed the Parcel Box Planner and saved the repaired page as final output 1.html. I preserved the original starter and the requirements brief. The result is one standalone local HTML file. It produces a synthetic packing plan without booking a shipment, sending a request or saving a record."
SAY	"I began by checking the packing policy and demonstrating two incorrect baseline results. The important distinction was between completely filled boxes and all the boxes needed. Rounding down is correct for full boxes. Rounding up is needed for the total box count when items remain. An exact fit still needs only the whole-number quotient."
SAY	"I changed four short arithmetic statements. The remainder operator now counts the items in a partial box. Total slots multiplies the box count by the capacity of each box. Unused slots subtracts the items from that capacity. Those units help explain why adding a box count to a capacity, or reversing the final subtraction, gives the wrong meaning."
SAY	"In Firefox, I completed all seven acceptance groups. The twenty-three-item order needed four boxes and left one slot unused. Twenty-four items fit exactly, with no partial box and no unused space. I also checked one item, a one-item box capacity and a box larger than the order. The ten-thousand-item case confirmed the same calculations at the largest permitted item count."
SAY	"The final group checked native browser validation. A missing item count, zero box capacity and a fractional item count were blocked before a new calculation ran. The old result table remained visible, so I distinguished that previous plan from a newly calculated result. I then restored twenty-three items and a capacity of six and confirmed that the expected plan returned."
SAY	"The main reasoning challenge was keeping a full-box count, a total-box count and empty capacity separate. The checks give confidence in this small arithmetic planner, but they do not prove that real items physically fit into real shipping boxes. A production packing system would also need dimensions, weight limits and suitable server validation. For this task, I checked the saved formulas against the written policy and kept the exact tested file for delivery."

16. Stop - about 29:30-29:45

SAY	"I am left-clicking TaskRecorder in the side Dock, then Stop Recording. I will wait for the archive to finish before closing the recorder."
ACTION	Stop the continuous take, wait until archiving finishes and preserve its original archive and matching video unchanged.

Optional error branches - not part of a successful main take

IF WRONG	If any displayed result fails its CHECK, do not read its success SAY. Read the next SAY, inspect the exact saved formulas and choose only a branch whose cause you actually confirm.
SAY	"The result table in Firefox does not match the requirements brief. I am returning to final output 1.html in Visual Studio Code on the left to compare lines 52 through 55 with the packing policy."
IF WRONG	Use the next SAY only if line 52 is actually wrong. Apply its exact EDIT statement above, preserve the newline and save.

SAY	"In Visual Studio Code, line 52 in final output 1.html does not calculate the remainder correctly. I am changing it to item count remainder box size, using the percent operator. I will save the file, reload the output in Firefox and repeat the failed group."
IF WRONG	Use the next SAY only if line 53 is actually wrong. Apply its exact EDIT statement above, preserve the newline and save.
SAY	"In Visual Studio Code, line 53 in final output 1.html does not round the total box count up correctly. I am changing it to Math.ceil of item count divided by box size. I will save, reload in Firefox and repeat the failed group."
IF WRONG	Use the next SAY only if line 54 is actually wrong. Apply its exact EDIT statement above, preserve the newline and save.
SAY	"In Visual Studio Code, line 54 in final output 1.html does not multiply the box count by its capacity. I am changing it to boxes needed times box size. I will save, reload in Firefox and repeat the failed group."
IF WRONG	Use the next SAY only if line 55 is actually wrong. Apply its exact EDIT statement above, preserve the newline and save.
SAY	"In Visual Studio Code, line 55 in final output 1.html subtracts the quantities incorrectly. I am changing it to total slots minus item count. I will save, reload in Firefox and repeat the failed group."
IF WRONG	A corrected test is checked with its original inputs and expected rows. Use its original field-entry SAY and CHECK. Read its success SAY only if it now matches. Never invent a cause. If no confirmed branch explains the problem, read the next SAY and stop the attempt to investigate off camera.
SAY	"I have an unexpected problem that I cannot explain with the checked calculation statements. I am stopping this attempt to investigate before recording again."

After recording - off screen

ACTION	Review the actual recording and its capture archive. Keep the original continuous video and archive unchanged. Supply the exact recorded final output 1.html and actual chronological app-switch order for metadata. Confirm that all P01-P07 checks were demonstrated. The eventual deliverables ZIP contains the two original inputs and the exact recorded output; metadata is uploaded separately. Run both AutoQC checks in the same approved Studio task.